

analysis

September 16, 2026

1 Poorman

1.1 Thresholds

Threshold functions are non-negative by definition. Formulas below are stated for the range where they are positive. Each should be read with a $\max(0, \dots)$ wrapper.

1.1.1 tow(2)

```
[1]: from games import make_tow
      from plotting import draw_game

      draw_game(make_tow(2))
```



$$T_{tow(2)}(1, B) = T_{tow(2)}(2, B) + B$$

$$T_{tow(2)}(2, B) = \max(0, B - 1)$$

```
[2]: from allpay import Player
      from games import make_tow
      from checks import assert_poorman_thresholds

      game = make_tow(2)
```

```

assert_poorman_thresholds(game, 1, lambda b: max(0, b - 1) + b, 100_000)
assert_poorman_thresholds(game, 2, lambda b: max(0, b - 1), 100_000)
print("OK")

```

OK

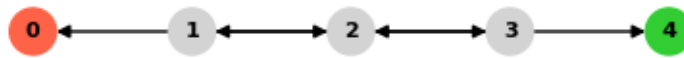
1.1.2 tow(3)

```

[3]: from games import make_tow
      from plotting import draw_game

      draw_game(make_tow(3))

```



$$T_{tow(3)}(1, B) = T_{tow(3)}(2, B) + B$$

$$T_{tow(3)}(2, B) = \lfloor (B - 1)\varphi \rfloor - 1 \quad *$$

$$T_{tow(3)}(3, B) = \max(0, B - 2)$$

* except when $B - 1$ is an even-indexed Fibonacci number F_{2k} , in which case $T_2(B) = F_{2k+1} - 1$

```

[4]: import math

      from allpay import Player
      from games import make_tow
      from checks import assert_poorman_thresholds

      PHI = (1 + math.sqrt(5)) / 2

      fibonacci = [0, 1]

```

```

while len(fibonacci) < 300:
    fibonacci.append(fibonacci[-1] + fibonacci[-2])
fibonacci_index = {f: i for i, f in enumerate(fibonacci)}

def t2(b):
    x = b - 1
    f_idx = fibonacci_index.get(x)
    if f_idx is not None and f_idx % 2 == 0 and f_idx > 0:
        return max(0, fibonacci[f_idx + 1] - 1)
    return max(0, math.floor(x * PHI) - 1)

game = make_tow(3, Player.P1)
assert_poorman_thresholds(game, 3, lambda b: max(0, b - 2), 100_000)
assert_poorman_thresholds(game, 2, t2, 100_000)
assert_poorman_thresholds(game, 1, lambda b: t2(b) + b, 100_000)
print("OK")

```

OK

1.1.3 race(a,b)

$$T_{\text{race}(a,b)}(B) = B + (a - 1) \left\lfloor \frac{B}{b} \right\rfloor - (b - 1)$$

```

[5]: from games import make_race
from checks import assert_poorman_thresholds

def race_closed_form(a, b):
    return lambda budget: max(0, budget + (a - 1) * (budget // b) - (b - 1))

pairs = [(a, b) for a in range(1, 10) for b in range(1, 10)]

for a, b in pairs:
    assert_poorman_thresholds(make_race(a, b), 0, race_closed_form(a, b), 1000)

print("OK")

```

OK

1.1.4 Threshold ratio

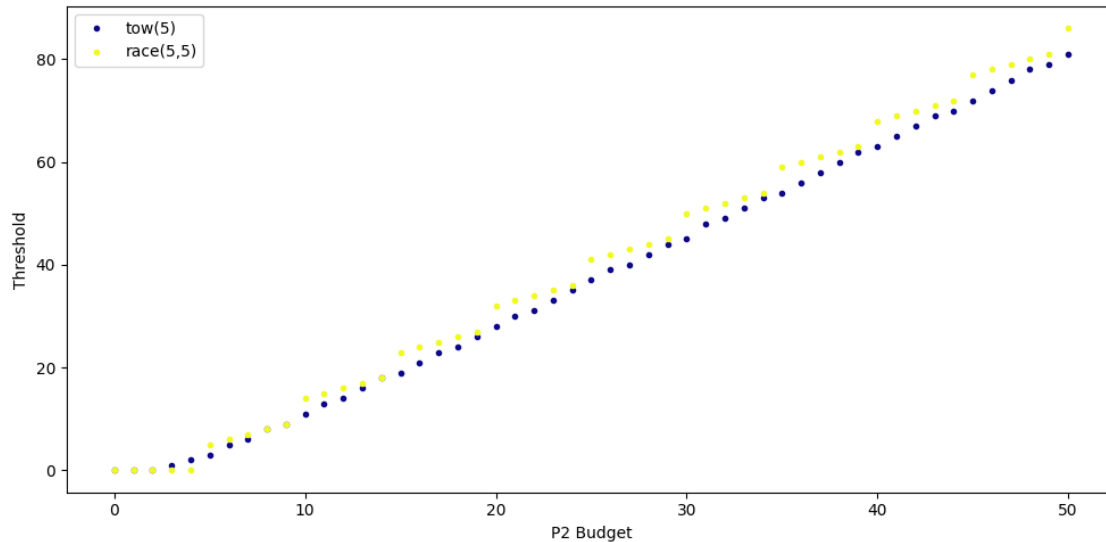
Conjecture: For every non-target vertex v , $\lim_{B \rightarrow \infty} \frac{T_v(B)}{B} \geq 1$

1.1.5 tow(n) vs race(n,n)

Observation: The thresholds of tow(n) (from the middle node) and race(n, n) are very similar.

```
[6]: from allpay import RootedGame
      from games import make_tow, make_race
      from plotting import plot_thresholds

      plot_thresholds([(RootedGame(make_tow(5), 3), ↵
        ↵"tow(5)"), (RootedGame(make_race(5, 5), 0), "race(5,5)"]], 50)
```

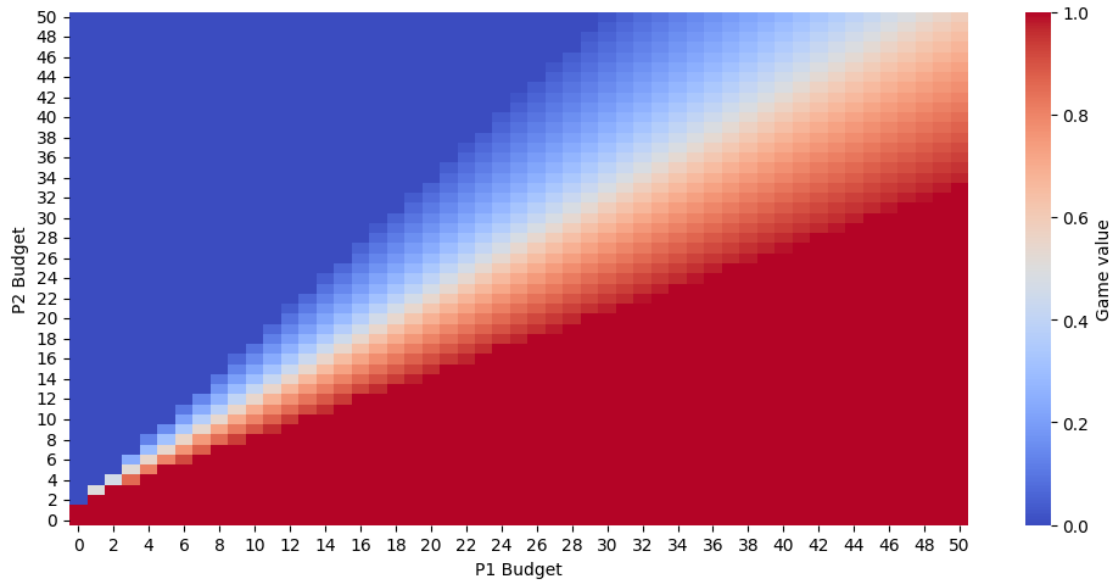


1.2 Pure vs Mixed Strategies

States where neither player has a pure winning strategy exist. In these states, mixed strategies are used. Therefore neither player can force a guaranteed win in these states, but we can still calculate the game value (P1's win probability under optimal mixed play). Below is presented the game value heatmap of tow(3) from the middle node.

```
[7]: from allpay import RootedGame
      from games import make_tow
      from plotting import plot_values_heatmap

      plot_values_heatmap(RootedGame(make_tow(3), 2), 50)
```

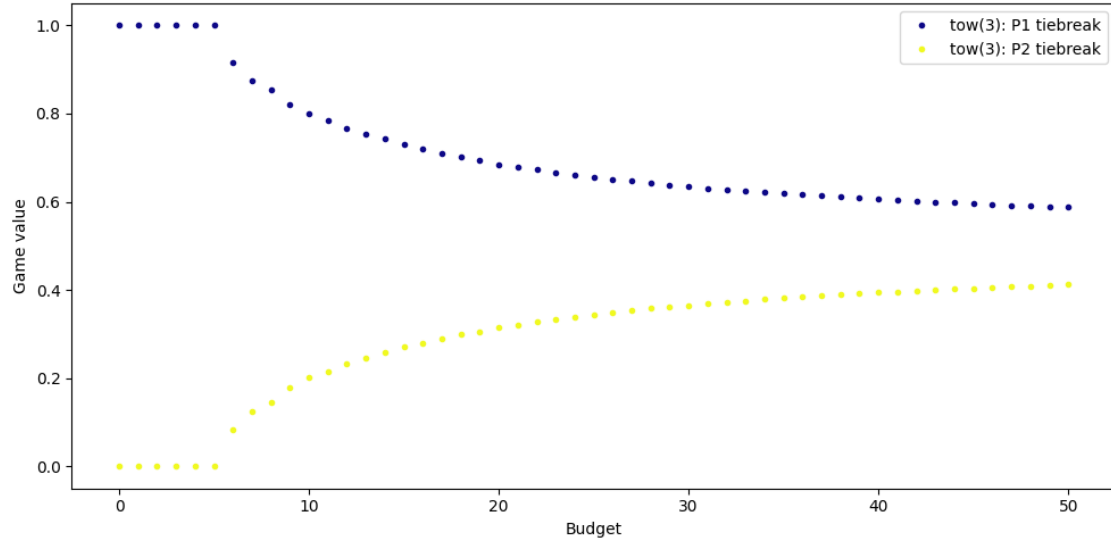


Conjecture: Whenever a player has a winning strategy from a state, a pure winning strategy from that state exists as well.

Conjecture: In a symmetric game, the game value with equal budgets approaches $1/2$ as the budget grows.

```
[8]: from allpay import Player
      from allpay import RootedGame
      from games import make_race, make_tow
      from plotting import plot_game_values

      plot_game_values([(RootedGame(make_tow(3), 2), "tow(3): P1_
        ↳tiebreak"),(RootedGame(make_tow(3, Player.P2), 2), "tow(3): P2_
        ↳tiebreak"),],50)
```

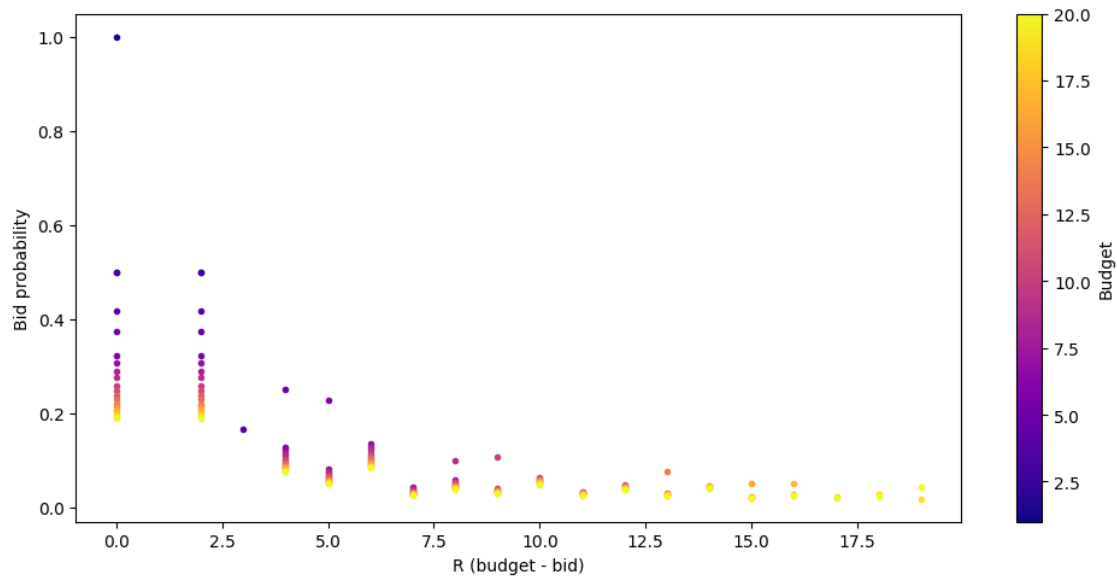


Observation: For a fixed remaining amount R , the weight the optimal strategy puts on leaving exactly R is non-increasing as the budget grows.

Below are presented the optimal strategies (by remaining budget) for tow(2), node 1, across different budgets.

```
[9]: from allpay import RootedGame
from games import make_tow
from plotting import plot_strategy_remaining

plot_strategy_remaining(RootedGame(make_tow(2), 1), 20)
```



2 Richman

2.1 Closed-form Solutions

2.1.1 tow(n), race(n, 1)

$$T_{\text{tow}(n)}(i, B) = T_{\text{race}(n-i+1, 1)}(B) = (2^{n-i+1} - 1) B$$

```
[10]: from allpay import Player
      from games import make_tow, make_race
      from checks import assert_richman_thresholds

      for n in range(1, 10):
          for i in range(1, n + 1):
              game = make_tow(n)
              assert_richman_thresholds(game, i, lambda b: (2**(n - i + 1) - 1) * b,
          ↪ if b >= i else 0, 1000)

      for a in range(1, 10):
          game = make_race(a, 1)
          assert_richman_thresholds(game, 0, lambda b: (2**a - 1) * b, 1000)

      print("OK")
```

OK